

SQL Server Practice Workbook

Intermediate-Level Tasks & Solutions — SalesDB Dataset

This workbook contains 12 intermediate-level T-SQL exercises built on the **SalesDB** database, covering joins, aggregation, subqueries, window functions, self-joins, and date functions. Each task includes a clear objective followed by a complete, ready-to-run T-SQL solution. Work through the tasks first before checking the solutions.

Database Schema Overview

Table	Key Columns
Sales.Customers	CustomerID (PK), FirstName, LastName, Country, Score
Sales.Employees	EmployeeID (PK), FirstName, LastName, Department, BirthDate, Gender, Salary, ManagerID
Sales.Products	ProductID (PK), Product, Category, Price
Sales.Orders	OrderID (PK), ProductID, CustomerID, SalesPersonID, OrderDate, ShipDate, OrderStatus, ShipAddress, BillAddress, Quantity, Sales, CreationTime
Sales.OrdersArchive	Same columns as Sales.Orders (historical/archived orders, no PK)

Task 1 — Total Sales per Customer

List every customer's full name, country, and their total Sales amount across all orders. Order the result from the highest total sales to the lowest.

SOLUTION

```
SELECT
    c.CustomerID,
    c.FirstName + ' ' + c.LastName AS CustomerName,
    c.Country,
    SUM(o.Sales) AS TotalSales
FROM Sales.Customers AS c
JOIN Sales.Orders AS o
    ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerID, c.FirstName, c.LastName, c.Country
ORDER BY TotalSales DESC;
```

Task 2 — Employees Who Earn More Than Their Manager

Find every employee whose Salary is greater than their manager's Salary. Show the employee name, employee salary, manager name, and manager salary.

SOLUTION

```
SELECT
    e.FirstName + ' ' + e.LastName AS EmployeeName,
    e.Salary AS EmployeeSalary,
    m.FirstName + ' ' + m.LastName AS ManagerName,
    m.Salary AS ManagerSalary
FROM Sales.Employees AS e
JOIN Sales.Employees AS m
    ON e.ManagerID = m.EmployeeID
WHERE e.Salary > m.Salary;
```

Task 3 — Rank Customers by Score Within Each Country

For each country, rank customers by Score in descending order (highest score = rank 1). Customers with equal scores should receive the same rank.

SOLUTION

```
SELECT
    CustomerID,
    FirstName + ' ' + LastName AS CustomerName,
    Country,
    Score,
    RANK() OVER (PARTITION BY Country ORDER BY Score DESC) AS ScoreRank
FROM Sales.Customers
ORDER BY Country, ScoreRank;
```

Task 4 — Products That Have Never Been Ordered

Return every product that does not appear in the Orders table at all.

SOLUTION

```
SELECT
```

```

        p.ProductID,
        p.Product,
        p.Category,
        p.Price
FROM Sales.Products AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM Sales.Orders AS o
    WHERE o.ProductID = p.ProductID
);

-- Equivalent LEFT JOIN version:
-- SELECT p.*
-- FROM Sales.Products AS p
-- LEFT JOIN Sales.Orders AS o ON o.ProductID = p.ProductID
-- WHERE o.OrderID IS NULL;

```

Task 5 — Monthly Sales Trend

Calculate the total Sales and the number of orders for each year and month, ordered chronologically.

SOLUTION

```

SELECT
    YEAR(OrderDate) AS OrderYear,
    MONTH(OrderDate) AS OrderMonth,
    COUNT(OrderID) AS TotalOrders,
    SUM(Sales) AS TotalSales
FROM Sales.Orders
WHERE OrderDate IS NOT NULL
GROUP BY YEAR(OrderDate), MONTH(OrderDate)
ORDER BY OrderYear, OrderMonth;

```

Task 6 — Running Total of Sales per Customer

For each customer, show every order with a running (cumulative) total of Sales, ordered by OrderDate.

SOLUTION

```

SELECT
    CustomerID,
    OrderID,
    OrderDate,
    Sales,
    SUM(Sales) OVER (
        PARTITION BY CustomerID
        ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS RunningTotalSales
FROM Sales.Orders
ORDER BY CustomerID, OrderDate;

```

Task 7 — Customers With an Above-Average Score

Return all customers whose Score is greater than the average Score across all customers.

SOLUTION

```

SELECT
    CustomerID,
    FirstName + ' ' + LastName AS CustomerName,
    Country,
    Score
FROM Sales.Customers
WHERE Score > (
    SELECT AVG(Score)
    FROM Sales.Customers
)
ORDER BY Score DESC;

```

Task 8 — Departments Paying Above the Company-Wide Average

Find each department whose average Salary is higher than the average Salary across the entire company. Show the department name and its average salary.

SOLUTION

```

SELECT
    Department,
    AVG(Salary) AS AvgDepartmentSalary
FROM Sales.Employees
GROUP BY Department
HAVING AVG(Salary) > (
    SELECT AVG(Salary)
    FROM Sales.Employees
)
ORDER BY AvgDepartmentSalary DESC;

```

Task 9 — Orders That Exist in Both Orders and OrdersArchive

Identify OrderIDs that appear in both Sales.Orders and Sales.OrdersArchive (i.e., orders that were archived but also still exist in the live table).

SOLUTION

```

SELECT OrderID
FROM Sales.Orders
INTERSECT
SELECT OrderID
FROM Sales.OrdersArchive;

-- Equivalent EXISTS version:
-- SELECT o.OrderID
-- FROM Sales.Orders AS o
-- WHERE EXISTS (
--     SELECT 1
--     FROM Sales.OrdersArchive AS a
--     WHERE a.OrderID = o.OrderID
-- );

```

Task 10 — Top 3 Best-Selling Products by Quantity

Find the top 3 products with the highest total Quantity sold. Use a window function so ties share the same rank.

SOLUTION

```
WITH ProductTotals AS (  
    SELECT  
        p.ProductID,  
        p.Product,  
        SUM(o.Quantity) AS TotalQuantity  
    FROM Sales.Products AS p  
    JOIN Sales.Orders AS o  
        ON o.ProductID = p.ProductID  
    GROUP BY p.ProductID, p.Product  
)  
SELECT  
    ProductID,  
    Product,  
    TotalQuantity,  
    DENSE_RANK() OVER (ORDER BY TotalQuantity DESC) AS QuantityRank  
FROM ProductTotals  
ORDER BY QuantityRank  
OFFSET 0 ROWS FETCH NEXT 3 ROWS ONLY;
```

Task 11 — Average Shipping Time per Sales Employee

For each SalesPersonID, calculate the average number of days between OrderDate and ShipDate. Round the result to 1 decimal place.

SOLUTION

```
SELECT  
    SalesPersonID,  
    ROUND(AVG(CAST(DATEDIFF(DAY, OrderDate, ShipDate) AS FLOAT)), 1) AS AvgShippingDays  
FROM Sales.Orders  
WHERE ShipDate IS NOT NULL  
    AND OrderDate IS NOT NULL  
GROUP BY SalesPersonID  
ORDER BY AvgShippingDays;
```

Task 12 — Customers With No Orders in the Last 6 Months

Return customers who have placed at least one order in the past, but none in the last 6 months relative to today's date.

SOLUTION

```
SELECT  
    c.CustomerID,  
    c.FirstName + ' ' + c.LastName AS CustomerName,  
    MAX(o.OrderDate) AS LastOrderDate  
FROM Sales.Customers AS c  
JOIN Sales.Orders AS o  
    ON o.CustomerID = c.CustomerID  
GROUP BY c.CustomerID, c.FirstName, c.LastName  
HAVING MAX(o.OrderDate) < DATEADD(MONTH, -6, CAST(GETDATE() AS DATE))  
ORDER BY LastOrderDate;
```